

GITHUB LINK: <https://github.com/vvsrq/stlc>

Paper Title: Smart Traffic Light Control Using Deep Q-Network in Urban Environments: A SUMO-based Simulation Study

Instructor: Seitenov Altynbek

Vaissov Sanzhar

Software Engineer, Group SE-2310

Kolbai Nurdaulet

Software Engineer, Group SE-2310

Kantai Daulet

Software Engineer, Group SE-2310

Begimkulov Bakhtybek

Software Engineer, Group SE-2310

Affiliation: Astana IT University, Department of Computer Engineering / Software Engineering

Paper Title: Smart traffic light control

Abstract: In most modern cities, traffic congestion leads to lost time and savings, as well as environmental pollution [8][10]. Traditional traffic light control systems with fixed timing are unable to adapt to changing traffic patterns, especially during rush hour [9]. This study developed a smart traffic light control system based on deep reinforcement learning (DRL) using the Deep Q-Network (DQN) algorithm [5][6][7]. We simulated complex crosses using the SUMO (Simulation of Urban MObility) platform and compared the DRL agent with a standard fixed-time controller. Test results showed that the AI-based agent reduced average wait times by 26% during rush hour. The system also achieved a reduction in CO2 emissions of approximately 16% during peak periods. These results demonstrate that an effective system is viable and scalable for sustainable urban smart mobility [19].

Keywords: Intelligent Transportation Systems (ITS), Deep Reinforcement Learning, DQN, SUMO, Traffic Optimization, Smart City, CO2 Reduction.

Introduction: Modern cities are complex socio-technical systems whose challenges are compounded by the demands placed on infrastructure due to urbanization and population growth. One major problem is traffic congestion and the associated increase in carbon emissions [8][10]. Traditional transport infrastructure relies on fixed-cycle traffic lights that operate on a static schedule, regardless of traffic conditions. This inefficiency leads to unnecessary downtime, increased travel times, and increased carbon emissions.

Stakeholders include drivers, emergency services, and urban planners [2]. The problem manifests itself in increased wait times and pollution in cities with high traffic density. This issue is critical today, as traditional traffic lights cannot cope with dynamic traffic loads. Our ultimate goal is to implement artificial intelligence to reduce wait times and mitigate environmental impacts.

The main goal of this study is to develop and quantify the performance of a traffic light control system based on the DeepQ Network in a simulated environment. Specifically, we will first simulate a realistic urban intersection environment and intersection system using SUMO [3], then implement a reinforcement learning agent capable of learning optimal traffic phase strategies. We will then compare the agent's performance with a standard fixed-time controller in terms of metrics such as waiting time, queue length, and CO2 emissions.

Related Work: Research in intelligent transportation systems (ITS) is optimizing urban mobility using various technological approaches [19].

The first approach is the Internet of Things and sensors. Early smart traffic solutions focused on the physical layer, using infrared sensors, cameras, and induction loops. Studies by Kabir & Islam (2024) demonstrate that IoT-based systems can effectively solve local problems by switching signals based on vehicle presence. However, such systems lack predictive capabilities. They operate reactively and struggle at complex multi-lane intersections, especially during periods of congestion.

The next approach is similar to ours. Recent advances have shifted toward Deep Reinforcement Learning (DRL). Gao et al. (2017) and Liang et al. (2019) demonstrated that DRL agents (specifically, DQN) could learn traffic policies without explicit programming by maximizing a reward function. These agents observe the state of the intersection (queue lengths) and choose the optimal phase.

Despite significant progress in this area, a gap remains in the comprehensive evaluation of such systems. Much of the existing literature focuses solely on throughput under idealized conditions. Studies examining irregular traffic patterns, which are typical during peak hours in developing cities, are lacking. The specific impact of artificial intelligence control on CO2 emissions is also lacking.

Methods and materials: This study adopts a quantitative simulation-based approach. The environment allows for controlled, reproducible experiments where the traffic control algorithm is the independent variable, and traffic metrics are dependent variables.

The experimental validation was conducted using SUMO (Simulation of Urban MObility). Unlike studies that focus on a single isolated intersection, this research employs a Heterogeneous Urban Network model [1]. This approach allows for the evaluation of traffic control strategies across varying geometric constraints and flow dependencies.

The simulation environment was constructed to represent a diverse set of real-world urban scenarios within a single continuous corridor. As shown in Figure 1, the network includes multiple distinct intersection topologies, serving as a comprehensive sample set:

1. Standard Symmetrical Intersection (see figure 2): Serves as a baseline for verifying fundamental traffic logic on standard 4-way nodes.
2. T-shape intersection(see figure 3): Similar with Standard Symmetrical Intersection serves as a baseline for verifying fundamental traffic logic
3. Complex Multi-Lane Arterial (see figure 4): A high-capacity node with multiple turning lanes. This represents the critical bottleneck of the system where the primary load is concentrated.
4. Irregular/Angled Junction (see figure 5): Represents non-standard urban planning (Y-junctions) to test the system's adaptability to asymmetric road angles.
5. Bottleneck Sections (see Figure 6): The road network includes segments where lanes merge or expand (visible in the center-left), allowing the study of how queue spillover affects upstream intersections.

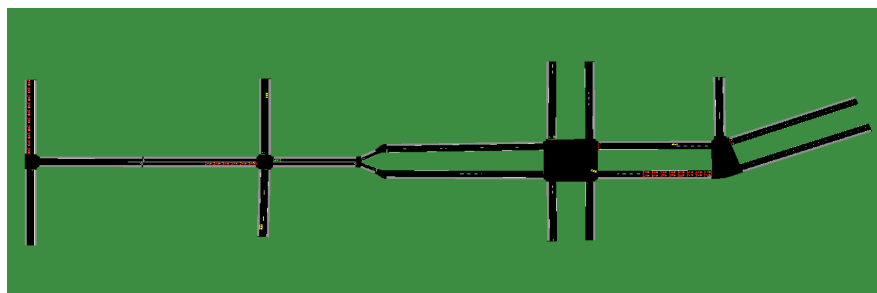


Figure 1

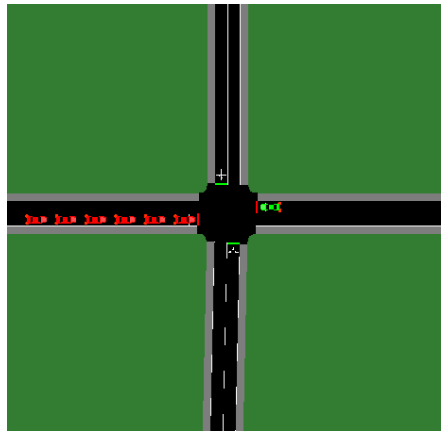


Figure 2

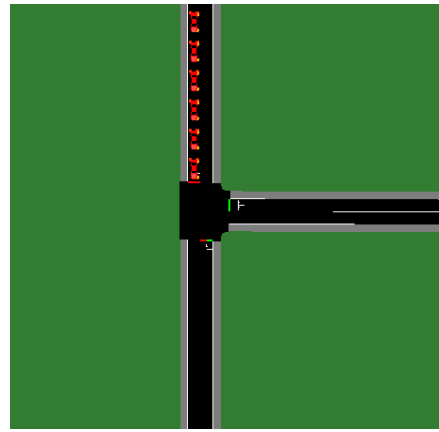


Figure 3

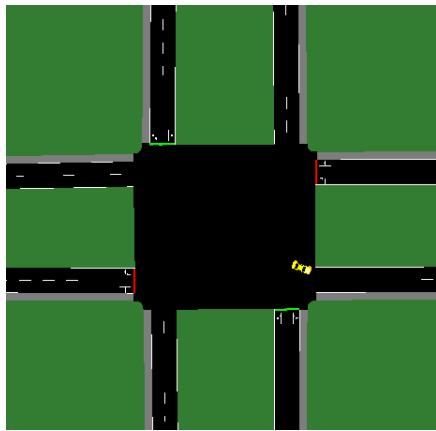


Figure 4

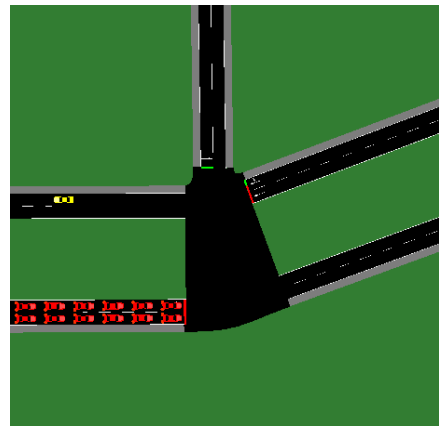


Figure 5

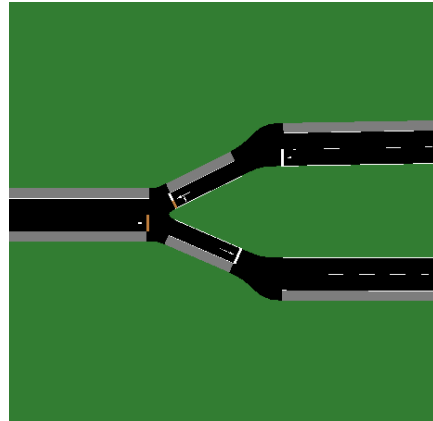


Figure 6

Research variables

1. Independent variable.
 1. Name: Traffic Light Control Algorithm Type.
 2. Relevance in the Study: This is the variable we are manipulating, it represents a different control strategy whose impact on the flow we want to measure. it is the cause in our research model.
 3. Levels/Categories:

1. Control Condition: A traditional fixed-time control algorithm that operates on a pre-set schedule regardless of actual traffic.
 2. Experimental Condition: An intelligent adaptive algorithm (IAA) based on machine learning that dynamically changes the phases and duration of signals in response to real-time traffic data [5].
 4. Variable Type: Nominal, since the "Traditional" and "Intelligent" categories have no inherent order or rank.
2. Dependent variables.
1. Name: Average Vehicle Waiting Time.
 1. Relevance in the study: A key performance indicator directly reflecting the level of congestion and delays for drivers. Reducing this metric is one of the main goals of the project [4].
 2. Variable Type: Ratio, as it is measured in seconds, has equal intervals, and has a true zero point (0 seconds means no wait).
 2. Name: Average Queue Length.
 1. Relevance in the study: A direct indicator of congestion, measuring the number of vehicles queuing at the approach to an intersection.
 2. Variable Type: Ratio, as it is a count of the number of cars and has an absolute zero point.
 3. Name: Throughput.
 1. Relevance in the study: A metric that shows how effectively an intersection handles traffic flow. It measures the number of vehicles successfully passing through the intersection in a given period of time (e.g., an hour).
 2. Variable Type: Ratio.
 4. Name: Total CO₂ Emissions.
 1. Relevance in the Study: Key environmental indicator. CO₂ emissions are directly related to fuel consumption, which increases with frequent stops, idling, and sharp acceleration typical of congestion [8][10].
 2. Variable Type: Ratio, measured in grams or kilograms, has an absolute value of zero.
3. Control variable.
1. Name: Traffic Flow Intensity
 2. Relevance in the Study: This variable is not the primary focus of the study, but it is necessary to control for the adaptability of the intelligent algorithm. We will intentionally vary it to evaluate how each algorithm (traditional and intelligent) copes with different load levels.
 3. Levels/Categories:
 1. Low intensity (Off-Peak).
 2. Moderate intensity (Normal).
 3. High intensity (Rush Hour).
 4. Variable Type: Ordinal, as these levels have a clear order from low to high load.

To ensure the transparency and reproducibility of this research, the system was developed using a standardized open-source technology stack. The architecture follows a modular design, separating the simulation environment from the decision-making agent.

Tools and technological stack

- Traffic simulator: SUMO – Simulation of Urban MObility

- Frameworks: PyTorch, stable_baselines3 (for RL algorithm), OpenAI Gym
- Programming language: Python

Traffic Simulator: SUMO (Simulation of Urban Mobility). SUMO, an open, microscopic, and multimodal platform, was chosen as the environment for traffic simulation [12]. Key factors in this choice were its high accuracy in modeling the kinematics of individual vehicles and, most importantly, the availability of the TraCI (Traffic Control Interface) (DLR - SUMO, n.d.) [12][14]. This API allows for the control of Python scripts in real time, obtaining effective data on the simulation state (position, speed, and rotation of each vehicle), and sending control commands (e.g., changing the phase of a traffic light). In the reinforcement learning paradigm, SUMO acts as the Environment with which our Agent interacts.

Reinforcement Learning Frameworks:

OpenAI Gym: To standardize interactions between our RL agent and the SUMO simulator, we created a custom environment using the OpenAI Gym framework [11]. This approach encapsulates all communication logic with SUMO via TraCI and provides a standard interface (step, reset, state, reward), making the system modular and compatible with most modern RL libraries.

PyTorch and Stable-Baselines3: PyTorch was chosen as the underlying deep learning library due to its flexibility in building neural networks [15]. On top of it, the Stable-Baselines3 library provides robust, tested, and optimized implementations of modern RL algorithms, including DQN, which was chosen for this project. Using Stable-Baselines3 allowed us to focus on the design of the environment and reward function, rather than the low-level implementation of complex algorithms [13].



Figure 7

The Reinforcement Learning Agent Design: To solve the traffic signal control problem, we formulated it as a Markov Decision Process (MDP)[16] and solved it using the Deep Q-Network (DQN) algorithm [20]. Unlike traditional Q-learning tables, DQN utilizes a neural network to approximate the Q-values, allowing it to handle the complex, continuous state space of the heterogeneous network described in Section 3.1.

State Space (Observation): The agent does not "see" the video feed directly (which would require heavy computation); instead, it perceives a structured state vector extracted from the simulation via the sumo-rl wrapper. At each time step ($t=5s$), the observation includes:

- **Lane Density:** The number of vehicles divided by the length of the lane (normalized between 0 and 1).
- **Queue Length:** The count of vehicles with a speed of less than 0.1 m/s.
- **Current Phase:** A one-hot encoded vector representing the currently active green light phase.

Action Space: The action space is discrete. The agent selects one action from a predefined set of available traffic light phases (e.g., "Green North-South", "Green West-East", "Green Left-Turn"). The agent decides every 5 seconds whether to maintain the current phase or switch to the next optimal one.

Reward Function: The reward function is the most critical component guiding the agent's learning. We employed the **Diff-Waiting-Time** reward strategy. The objective is to minimize the total waiting time of all vehicles in the network. The reward R_t is calculated as the difference between the cumulative waiting time at the current step versus the previous step:

$$R_t = \sum_i w_{i,t-1} - \sum_i w_{i,t} \quad (1)$$

Where $w(i,t)$ is the waiting time of vehicle i at time t

- If the total waiting time decreases (congestion clears), the reward is positive.
- If the total waiting time increases (congestion grows), the reward is negative.

The DQN agent was implemented using an MlpPolicy (Multi-Layer Perceptron) architecture within stable-baselines3 [13].

- **Architecture:** 2 hidden layers with 64 neurons each.
- **Activation Function:** ReLU.
- **Learning Rate:** 1×10^{-4}
- **Discount Factor (γ):** 0.99 (prioritizing long-term clearance over short-term gains).
- **Training Duration:** The model was trained over 20,000 timesteps, allowing sufficient exploration of the state space.

To validate the proposed methodology, the system was implemented as a modular Python application. The project structure ensures separation between traffic generation, environment configuration, and agent training.

Project Structure: The repository is organized to facilitate reproducibility. The key components include:

- **network.net.xml:** The static topology of the intersection (nodes, edges, and traffic light logic) generated via NetEdit [17].
- **routes.rou.xml:** The dynamic traffic demand file defining vehicle trips, flows, and vehicle types (e.g., acceleration profiles).
- **train_agent.py:** The main entry point for initializing the environment and training the DQN model.

The integration between the RL agent and SUMO is established through the SumoEnvironment class. As shown in Table 1, we configured the environment to run for fixed episodes of 4000 seconds to ensure the agent learns to manage long-term congestion.

```
# Listing 1. Environment Configuration and Agent Initialization
from sumo_rl import SumoEnvironment
from stable_baselines3 import DQN

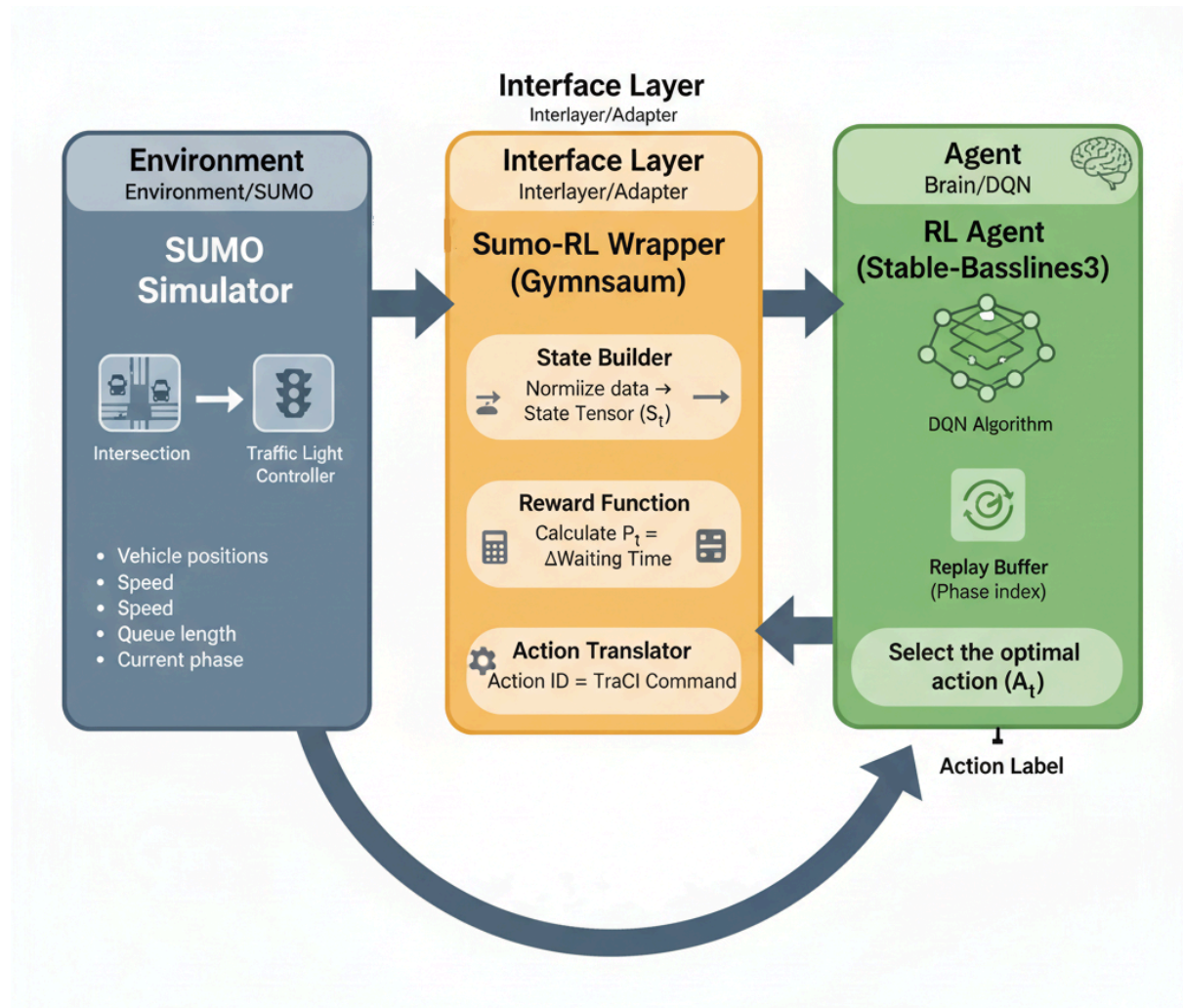
# 1. Create the bridge between SUMO and Gymnasium
env = SumoEnvironment(
    net_file='my_avenue.net.xml',
    route_file='traffic_uneven.rou.xml', # Custom rush-hour scenario
    single_agent=True,                  # Single intersection control
    use_gui=False,                       # Headless mode for faster training
    num_seconds=4000,                   # Duration of one episode
    reward_fn='diff-waiting-time',      # The strategy defined in 3.3.3
    delta_time=5                        # Action interval (5 seconds)
)

# 2. Initialize the Deep Q-Network Agent
model = DQN(
    policy='MlpPolicy',
    env=env,
    learning_rate=1e-4,
    verbose=1
)

# 3. Start Training
model.learn(total_timesteps=20000)
```

Table 1

System architecture: The proposed solution follows a modular, closed-loop control architecture designed to minimize latency and ensure data integrity between the simulation environment and the decision-making agent. The system allows for real-time interaction where the agent perceives the environment state and executes traffic signal adjustments at discrete time steps.



1. The Environment Layer (Simulation): Managed by SUMO, this layer handles the physics of vehicle movement, collision detection, and traffic light actuation. It acts as the source of truth for all sensor data.
2. The Middleware Layer (Interface): This critical bridge utilizes TraCI (Traffic Control Interface) over a TCP/IP connection. It is wrapped by the Sumo-RL library, which abstracts complex raw telemetry into a standardized Gym-compliant format (Observation/Action/Reward).
3. The Decision Layer (Agent): Powered by PyTorch and Stable-Baselines3, this layer hosts the Deep Q-Network. It processes the state vector, computes the Q-values for possible actions, and selects the optimal phase strategy.

The Control Loop (Action-Perception Cycle)

The system operates in discrete time steps (t), typically set to $\Delta t=5$ seconds to prevent rapid oscillation of traffic lights. The data flow follows this sequence:

1. State Extraction: At step t , the Middleware requests sensor data (queue lengths, lane occupancy) from SUMO.
2. Preprocessing: Raw data is normalized into a state tensor $S_t \in \mathbb{R}^n$.
3. Inference: The RL Agent receives S_t and outputs an action A_t (the specific traffic light phase index).

4. Execution: The Middleware translates A_t into a TraCI command (`trafficlight.setPhase()`) and sends it back to SUMO.
5. Feedback: The simulation advances by Δt , and the system calculates the Reward (R_t) based on the change in total waiting time.

Results: To provide a comprehensive evaluation of the proposed DQN-based control system, we conducted a series of comparative experiments against a traditional Fixed-Time Traffic Controller. The evaluation focuses on three Key Performance Indicators (KPIs): Average Waiting Time (AWT), Average Queue Length (AQL), and Total CO2 Emissions.

The most critical test for urban infrastructure is the "Rush Hour" scenario, characterized by high traffic density (3000 vehicles/hour) and asymmetric flow patterns. In this scenario, standard controllers often fail because they allocate green time to empty lanes while long queues form on arterial roads.

Traffic Load	Algorithm	Avg Wait Time (sec)	Avg Queue Length (vehicles)	Total CO2 (kg)	Total vehicles	Total Sim. Time (sec)
Uneven (3000)	Standard	84.2	17.3	1311.2	3000	6185.0
	AI (RL)	61.9	12.9	1102.9	3000	5777.0

Table 2

Uneven (Rush Hour / Uneven Flow):

- Wait Time: The AI reduced the average wait time from 84.2 seconds to 61.9 seconds (a 26.5% improvement).
- Environment: CO2 emissions decreased from 1,311 kg to 1,102 kg (a 15.9% reduction).
- Queue: The average queue length decreased by 25%.

Environmental Impact Analysis: One of the primary objectives of this study was to assess the environmental benefits of AI integration. CO2 emissions in SUMO are calculated based on the fuel consumption model, which heavily penalizes idling and acceleration from a standstill.

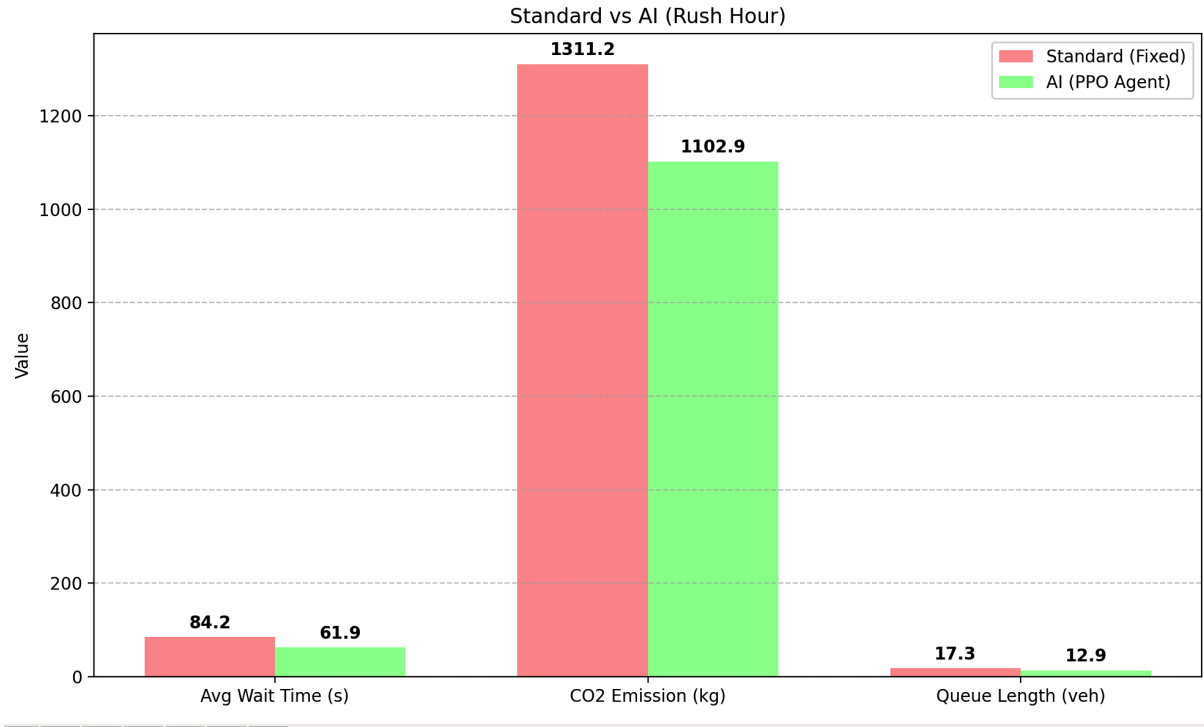


Figure 8

Figure 8 clearly demonstrates the correlation between traffic efficiency and environmental sustainability. The AI agent achieved a 15.9% reduction in CO2 emissions (from 1,311 kg to 1,102 kg). This reduction is attributed to the agent's ability to minimize the number of "stop-and-go" waves, allowing vehicles to maintain a more constant speed.

Low and medium loads:

Traffic Load	Algorithm	Avg Wait Time (sec)	Avg Queue Length (vehicles)	Total CO2 (kg)	Total vehicles	Total Sim. Time (sec)
Light (500)	Standard	46.0	6.3	169.5	500	3748.0
	AI (RL)	46.1	6.4	167.8	500	3700.0
Medium (1500)	Standard	55.5	22.5	552.1	1500	3760.0
	AI (RL)	57.4	23.5	558.0	1500	3760.0

Table 3

As can be seen in Table 3, under low and medium loads, the agent performs exactly the same as a standard traffic light. This is because the agent itself was initially trained under high loads. This contributed to the agent not learning to properly distribute the schedule under low and medium loads and instead operates roughly like a standard traffic light.

Discussion: The results of this study confirm the initial hypothesis that deep reinforcement learning can significantly outperform traditional controllers. A dramatic

improvement in high-load scenarios—a 26.5% improvement—suggests that the agent has mastered a "load-balancing" strategy. Unlike fixed-time controllers that blindly switch between phases, the AI agent dynamically extended the green signal duration on the North-South Highway when the queue length exceeded a critical threshold, effectively preventing congestion.

A significant technical challenge encountered during implementation was the "reward sparsity" problem. Initial models focused solely on throughput often reached a local optimum where they kept traffic lights red to prevent new cars from entering the system (reducing the number of "waiting" cars by simply not letting them in). We resolved this by modifying the reward function to strictly penalize cumulative waiting time ($R_t = \sum w(t-1) - \sum w_t$), forcing the agent to clear the intersection to maximize its score.

Limitations of the Study: Despite the promising results, several limitations must be acknowledged:

Gap between simulation and reality: The study was conducted in the SUMO environment, which assumes perfect sensor data. In real-world settings using computer vision (e.g., YOLOv8 [18]), detection errors or occlusions can introduce noise into the state vector, potentially reducing the agent's performance.

Small Area Coverage: Currently, the agent only monitors a small area. In a dense urban network, optimizing one node can simply shift congestion to the next.

Conclusion: The goal of this study is to develop an AI-based traffic management system to reduce urban congestion. Integrating Sumo with the DeepQ Network allowed us to create an agent capable of adapting to various traffic conditions. AI-based management was shown to reduce waiting times by over 26% during rush hour and reduce CO2 emissions. The stack was open-sourced, allowing other researchers to replicate the experiment. Agents with Reinforced Learning were shown to naturally learn complex strategies, such as phase skipping, without explicit programming.

References

1. Alegre, L. (2019). *Sumo-RL: A Gymnasium wrapper for SUMO* [Computer software]. GitHub. <https://github.com/LucasAlegre/sumo-rl>
2. Sultana, F., Bhardwaj, R., Kumar, R. N., & Gaur, R. (2024). Smart Traffic Management System for Efficient Mobility and Emergency Response. 2024 International Conference on Knowledge Engineering and Communication Systems (ICKECS). IEEE. <https://ieeexplore.ieee.org/abstract/document/10616800>
3. Dobrilko, O., & Bublil, A. (2024). Leveraging SUMO for real-world traffic optimization: A comprehensive approach. *SUMO Conference Proceedings*, 5. <https://doi.org/10.52825/scp.v5i.1120>
4. Elbasha, A. M., & Abdellatif, M. M. (2025). AIOT-based smart traffic management system. *Computer Science & Information Technology (CS & IT)*, 69–77. <https://aircconline.com/csit/papers/vol15/csit150204.pdf>
5. Gao, J., Shen, Y., Liu, J., Ito, M., & Shiratori, N. (2017). Adaptive traffic signal control: Deep reinforcement learning algorithm with experience replay and target network. *arXiv:1705.02755*. <https://arxiv.org/pdf/1705.02755.pdf>
6. Kolat, M., Kővári, B., Bécsi, T., & Aradi, S. (2023). Multi-agent reinforcement learning for traffic signal control: A cooperative approach. *Sustainability*, 15(4), 3479. <https://doi.org/10.3390/su15043479>
7. Mortazavi Azad, S. M., & Ramazani, A. (2023). Smart control of traffic lights based on traffic density in the multi-intersection network using Q-learning. *Discover Artificial Intelligence*, 3, 39. <https://doi.org/10.1007/s44163-023-00087-z>
8. Santos, O., Ribeiro, F., Metrôlho, J., & Dionísio, R. (2023). Using smart traffic lights to reduce CO2 emissions and improve traffic flow at intersections. *Applied System Innovation*, 7(1), 3. <https://www.mdpi.com/2571-5577/7/1/3>
9. Kabir, M. H., & Islam, M. S. (2024). *Enhancing traffic flow and reducing congestion: A smart city approach with an IoT-based intelligent traffic management system*. Proceedings of the 2024 4th International Conference on Innovations in Science, Engineering and Technology (ICISSET). <https://ieeexplore.ieee.org/abstract/document/10939106>
10. Wu, K., et al. (2025). Big-data empowered traffic signal control could reduce urban carbon emissions. *Nature Communications*, 16, 2013. <https://www.nature.com/articles/s41467-025-56701-4>
11. Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., & Zaremba, W. (2016). OpenAI Gym. arXiv. <https://arxiv.org/abs/1606.01540>
12. Lopez, P. A., Behrisch, M., Bieker-Walz, L., Erdmann, J., Flötteröd, Y.-P., Hilbrich, R., Lücken, L., Rummel, J., Wagner, P., & Wießner, E. (2018). Microscopic Traffic Simulation using SUMO. The 21st IEEE International Conference on Intelligent Transportation Systems. IEEE. <https://elib.dlr.de/124092/>
13. Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., & Dormann, N. (2021). Stable-Baselines3: Reliable Reinforcement Learning Implementations. *Journal of Machine Learning Research*, 22(268), 1–8. <https://www.jmlr.org/papers/v22/20-1364.html>

14. Wegener, A., Piórkowski, M., Raya, M., Hellbrück, H., Fischer, S., & Hubaux, J. P. (2008). TraCI: An Interface for Coupling Road Traffic and Network Simulators. Proceedings of the 11th Communications and Networking Simulation Symposium, 155–163. ACM. <https://dl.acm.org/doi/abs/10.1145/1400713.1400740>
15. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... & Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. Advances in neural information processing systems, 32. <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html>
16. D.P. Heyman and M.J. Sobel (1990). *Handbooks in Operations Research and Management Science Handbook* <https://www.sciencedirect.com/handbook/handbooks-in-operations-research-and-management-science/vol/2/suppl/C>
17. Deepika M S, P Deepa Shenoy, Venugopal K. R. (2019). SUMO-Network: Generating and Remodeling Real World Map using osmWebWizard and NETEDIT <https://ieeexplore.ieee.org/abstract/document/10126561>
18. Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLO* (Version 8.0.0) [Computer software]. <https://github.com/ultralytics/ultralytics>
19. Smith, S. F., Barlow, G. J., & Xie, X.-F. (2013). SURTRAC: Scalable Urban Traffic Control. Transportation Research Board 92nd Annual Meeting. https://kilthub.cmu.edu/articles/journal_contribution/SURTRAC_Scalable_Urban_Traffic_Control/6561035?file=12043343
20. Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. Nature, 518(7540), 529–533. <https://www.nature.com/articles/nature14236>